

Laboratorio 3

Procesamiento distribuido con Apache Spark

Paradigmas 2026 - FAMAF

Fecha límite de entrega: Viernes 11 de junio de 2026

Evaluación

La evaluación de este laboratorio es oral. Se va a pedir que expliquen partes de su código y defiendan algunas de las justificaciones escritas en [Informe.md](#), para verificar que las comprenden y no son simplemente texto generado. Prepárense para responder preguntas sobre cualquier decisión de diseño que hayan o no documentado.

Objetivo

El objetivo de este laboratorio es construir, usando Apache Spark, un programa que integre todas las funcionalidades desarrolladas en los laboratorios anteriores:

1. Leer suscripciones a partir de un archivo json.
2. Conectarse a múltiples feeds de Reddit y descargar los posts.
3. Imprimir estadísticas sobre los posts descargados
4. Extraer entidades nombradas del texto de cada post.
5. Clasificar cada entidad por tipo (persona, organización, lugar, etc.).
6. Imprimir conteo de entidades por tipo y por entidad.

Lo que cambia respecto a los laboratorios anteriores no es la lógica del problema sino cómo se ejecuta: en lugar de procesar los feeds uno por uno, un proceso coordinador (llamado *driver*) distribuye el trabajo se distribuye entre múltiples procesos (llamados

workers) usando Spark. El programa debe poder escalar a un número grande de feeds sin cambiar el código.

Metodología de trabajo

Les proveemos un esqueleto con una implementación en funcionamiento (pero sin manejo de errores) que no utiliza paralelización. La misma sigue los paradigmas de programación funcional y orientada a objetos.

Se espera que la salida de su solución final tenga exactamente el mismo formato que la salida que produce este esqueleto, pero que lo implementen usando la concurrencia que provee Scala, de forma que al ejecutarse tarde mucho menos tiempo para inputs grandes.

Existen dos formas de resolver este laboratorio:

1. Tomando como referencia el código del esqueleto y construyendo progresivamente su solución desde cero.
2. Modificando el esqueleto “in place”. En este caso, después de terminar cada ejercicio es necesario llamar a `collect` para traer los datos obtenidos de los *workers* al *driver* antes de continuar con el siguiente ejercicio.

Queda a su criterio qué metodología eligen.

Ejercicio 1 — Identificar las regiones paralelizables

Este ejercicio es de análisis. No requiere escribir código nuevo.

Un framework de computación distribuida como Spark impone una inversión de control: el desarrollador no escribe el código que coordina los *workers* ni decide cómo se distribuyen los datos. En cambio, expresa el cómputo usando las abstracciones que el framework provee (`map`, `flatMap`, `reduceByKey`, etc.) y Spark se encarga del resto. El punto caliente (*hot spot*) donde el desarrollador interviene es la función que se pasa a cada transformación; todo lo demás (scheduling, serialización, tolerancia a fallos) queda bajo control del framework.

Para poder usar esas abstracciones, primero hay que identificar qué partes del pipeline existente se corresponden con ellas. Respondan las siguientes preguntas en [Informe.md](#):

a) Dibujen el diagrama de flujo de los pasos que tiene que hacer su programa (conexión, descarga, extracción de entidades, clasificación, conteo, ranking) como un grafo de dependencias (que seguramente será algo muy parecido a una secuencia). Cada uno de los pasos será una acción o transformación que realiza un *worker* o el *driver*. La conexión entre un paso A y un paso B es el output de A y el input de B. Explícite el tipo en Scala de cada conexión.

b) Para cada paso del pipeline, determinen si puede expresarse como una de las abstracciones de Spark:

- **map**: transforma cada elemento en exactamente un resultado. Aplicable cuando cada tarea es independiente y produce exactamente una salida.
- **flatMap**: transforma cada elemento en cero o más resultados. Aplicable cuando cada tarea es independiente pero puede producir una cantidad variable de salidas.
- **reduceByKey** u otra reducción: combina múltiples elementos en uno agrupando por clave. Aplicable cuando el resultado depende de *todos* los elementos, no de uno solo.

¿Hay algún paso del pipeline que no encaje en ninguna de estas abstracciones? ¿Por qué?

c) Las reducciones constituyen una barrera de sincronización: ningún worker puede producir el resultado final hasta que todos hayan terminado su parte. Identifiquen qué pasos del pipeline son barreras y cuáles pueden ejecutarse de forma completamente independiente entre workers.

d) El mecanismo de extensión (*extension point*) de Spark es la función que el desarrollador le pasa a cada transformación. ¿Qué restricciones impone Spark sobre

esas funciones para que puedan ejecutarse en un entorno distribuido? Piensen en serialización, estado compartido y efectos secundarios.

Resultado esperado: respuestas a los cuatro puntos en [Informe.md](#).

Ejercicio 2 — Paralelizar la descarga de feeds

Concepto: `flatMap` sobre un RDD (*Resilient Distributed Database*, una estructura de datos inmutable) de URLs.

En la clase principal `Main` creen una `SparkSession` configurada en modo local:

```
None
val spark = SparkSession.builder()
  .appName("RedditNER")
  .master("local[*]")
  .getOrCreate()
val sc = spark.sparkContext
```

Al agregar Spark, tendrán que agregarlo como dependencia y posiblemente cambiar el comando con el que lo ejecutan para que tenga en cuenta la versión de Java correcta. Para más información ver el ejemplo `hello_world` provisto.

Sobre esa sesión, implementen lo siguiente:

- a) Lean las suscripciones de `subscriptions.json` y cárguelas en un RDD con `sc.parallelize(subscriptions)`.
- b) Apliquen un `flatMap` sobre el RDD de URLs que, para cada URL, descargue el feed y devuelva un iterador de posts. El resultado debe ser un `RDD[Post]` con todos los posts de todos los feeds donde `title` y `selftext` son ambos no vacíos.
- c) Impriman cuántos feeds se descargaron y cuántos fallaron, cuántos posts se descargaron y cuántos fallaron, cuántos fueron filtrados, y el largo promedio en caracteres de los posts filtrados.

d) Si no hay posts luego que se han filtrado los no vacíos, el programa debe imprimir `Error: No valid posts downloaded after filtering` y terminar

Al escribir el `flatMap`, manejen los errores dentro de la función de forma que un fallo no cancele el procesamiento del resto. En `Informe.md` expliquen qué pasaría si dejaran propagar la excepción.

Resultado esperado: programa que imprime el total de posts descargados al ejecutarse en el mismo formato que el esqueleto y maneja los siguiente casos de error.

Casos de error:

- Si un subscription no se puede parsear porque le falta el campo name o url, imprimir por cada una de ellas `Warning: Skipping malformed subscription (missing 'name' or 'url' field)`
- Si no existe el archivo de suscripciones, imprimir `Error: Could not load $filePath - file not found`
- Si no se puede parsear el archivo de suscripciones, imprimir `Error: Could not load $filePath - invalid JSON format`
- Si no hay suscripciones para procesar, imprimir `Error: No valid subscriptions found` y salir del programa.
- Si no existe el directorio de dictionary, `"Error: entities directory '$entitiesDir' not found"`
- Por cada archivo de entidad no encontrado o no leído correctamente, imprimir `println(s"Warning: Could not load $entitiesDir/people.txt")`
- Si no se puede parsear un post de una suscripción, por cada uno imprimir `"Warning: Failed to parse posts from '${subscription.name}' (${subscription.url})"`
- Si no se puede hacer fetch de una url, `"Warning: Failed to download from '${subscription.name}' (${subscription.url})"`

Nota: los prints sólo se verán si ejecutan Spark en modo local. Si lo ejecutan distribuido, los prints deben ser logs y se pueden ver a través de Spark UI.

Ejercicio 3 — Paralelizar el cómputo de entidades nombradas

Concepto: el patrón Map-Reduce aplicado al conteo de entidades.

El conteo de entidades nombradas es el caso de uso canónico de Map-Reduce. La fase **map** produce pares clave-valor de forma independiente por cada post; la fase **reduce** agrega los valores de la misma clave a través de todos los workers. El desarrollador solo provee las funciones en los puntos de extensión del framework; Spark se encarga de agrupar, shufflear y combinar.

Sobre el `RDD[Post]` del ejercicio anterior, construyan el siguiente pipeline encadenado:

- a) **flatMap**: para cada post, extraer sus entidades nombradas y devolver un iterador de `NamedEntity`.
- b) **map**: convertir cada `NamedEntity` en un par `(tipo, nombre)` como clave y `1` como valor, es decir, `((tipo, nombre), 1)`.
- c) **reduceByKey**: sumar los valores de cada clave para obtener el conteo total por entidad.
- d) Ordenen los resultados por conteo descendente y por tipo, y muestren el resultado por pantalla con el formato:

None

[TIPO] Entidad: N apariciones

En `Informe.md` respondan:

- **reduceByKey** es una barrera de sincronización. ¿Qué ocurre en el cluster en ese punto? ¿Por qué es inevitable para este problema?
- ¿Qué restricciones debe cumplir la función que se le pasa a **reduceByKey**? Piensen en conmutatividad y asociatividad.

- ¿Dónde se hace la lectura del diccionario de entidades? ¿En el driver o los workers?

Resultado esperado: pipeline completo que imprime las entidades clasificadas, contadas y rankeadas.

Ejercicio 4 — Monitoreo del éxito de las tareas

Concepto: Accumulators y Spark UI.

Un programa distribuido necesita observabilidad: saber cuánto trabajo se realizó, cuánto falló y cómo se distribuyó la carga. Spark provee dos mecanismos complementarios para esto. Los Accumulators son variables que los workers solo pueden incrementar y el driver solo puede leer; son el único mecanismo seguro para que un worker comunique información al driver sin romper el modelo funcional del pipeline. La Spark UI muestra en tiempo real el grafo de ejecución, el progreso de cada stage y las estadísticas de los workers.

a) Agreguen los siguientes Accumulators al pipeline:

- Número de feeds descargados con éxito.
- Número de feeds que fallaron (timeout, error HTTP, contenido inválido).
- Número de posts descargados en total.
- Número de posts descartados por tener texto nulo o vacío.

Impriman los cuatro valores por pantalla luego de la acción terminal del pipeline.

b) Utilicen estos acumuladores para calcular las estadísticas que se muestran por pantalla.

c) Midan el tiempo total de ejecución del pipeline con `System.currentTimeMillis()` antes y después de cada acción terminal e impriman la duración en segundos. Pista: son al menos dos puntos dentro de la ejecución del pipeline.

En [Informe.md](#) respondan:

- ¿Por qué los Accumulators solo deben usarse para métricas y no para tomar decisiones lógicas dentro de las etapas distribuidas del pipeline? ¿En qué situación un Accumulator puede dar un valor incorrecto?
- ¿En qué momento del pipeline está disponible el valor de un Accumulator para ser leído por el driver?
- Comparen el tiempo que tarda cada etapa del pipeline que midieron en la versión no paralelizada y la versión con Spark. ¿Qué conclusiones pueden sacar? Para la cantidad de datos que estamos trabajando, ¿se aprecia la diferencia? Justifique por qué. Nota: La comparación debe realizarse en ejecuciones sobre la misma computadora y la misma conexión a internet.

Resultado esperado: pipeline con los cuatro Accumulators, medición de tiempo, y captura de la Spark UI con su análisis en [Informe.md](#).

Ejercicio 5 — Acceso a datos y estadísticas del resultado

Concepto: persistencia de RDDs con `cache()` y push-down de operaciones a los workers.

Hasta ahora el pipeline ejecuta múltiples acciones en forma de consultas sobre los RDD e imprime resultados parciales. Adicionalmente, puede terminar preventivamente si no se encuentran posts válidos. Sin persistencia, cada acción recomputa todo el pipeline desde el principio, incluyendo las descargas HTTP. Con `.cache()`, cada RDD se materializa una sola vez y las consultas siguientes se leen de memoria.

- a) Identifiquen en qué puntos del código se están recomputando operaciones innecesariamente.
- b) Llamen a `.cache()` para cada RDD que se compute más de una vez.

c) Llamen a `unpersist()` donde corresponda, teniendo en cuenta los casos de error.

En `Informe.md` respondan:

- ¿Qué ocurriría si no llamaran a `cache()`? ¿Cuántas veces se ejecutaría la descarga de feeds?
- ¿Por qué es incorrecto llamar a `collect()` entre los pasos a) y b) del ejercicio 3 y luego continuar el pipeline? ¿Qué consecuencia tiene sobre la distribución del trabajo?
- `cache()` es también lazy. ¿En qué momento se almacena realmente el RDD en memoria?

Resultado esperado: programa donde cada operación del pipeline se ejecuta sólo una vez

Cómo correr el laboratorio

Utilizando un servidor local

Para evitar tener que realizar consultas a Reddit mientras se testea el programa, se puede utilizar un mock que simule la API de Reddit con un servidor local que devuelve posts estáticos.

En una terminal, desde la carpeta `reddit-mock/`:

```
None  
sbt run
```

Deberían ver:

```
None  
Fake Reddit API running on http://localhost:8123  
Press Ctrl+C (or stop the process) to shut down.
```

El servidor queda corriendo en el puerto 8123 y sirve los posts de los subreddits `r/scala`, `r/programming` y `r/learnpython` desde archivos JSON locales.

Para determinar si el programa realiza consultas a la API de Reddit o al mock local, sólo se debe cambiar la url en la suscripción.

Prerequisito: Apache Spark

Para instalar spark, solo es necesario agregarlo como una dependencia al archivo `build.sbt`.

Configuración de Java para Spark con Scala/SBT

Actualmente, Apache Spark 3.x no presenta compatibilidad con versiones de Java de vanguardia como la 24 o 25. Al intentar ejecutar el framework bajo estos entornos, es común encontrarse con excepciones del tipo:

None

```
java.lang.UnsupportedOperationException: getSubject is not supported
```

o bien:

None

```
IllegalAccessError: cannot access class sun.nio.ch.DirectBuffer
```

Para subsanar esto, la recomendación técnica es emplear Java 17 durante la ejecución de Spark.

Cabe destacar que no resulta necesario prescindir de Java 24/25 ni alterar la configuración global del sistema operativo. Es factible preservar la versión más reciente como predeterminada y restringir el uso de Java 17 exclusivamente al ámbito de este proyecto.

Linux (Ubuntu/Debian)

1. Instalación de Java 17

Shell

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk
```

Validar la instalación mediante:

Shell

```
java -version
```

2. Ejecución con Java 17

Shell

```
JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 \
```

```
PATH="$JAVA_HOME/bin:$PATH" \
```

```
SBT_OPTS="--add-exports=java.base/sun.nio.ch=ALL-UNNAMED" \
```

```
sbt run
```

Si el directorio varía según la distribución, pueden localizar el path correcto con:

Shell

```
update-alternatives --config java
```

o alternatively:

Shell

```
readlink -f $(which java)
```

macOS

1. Instalación de Java 17

En sistemas con Homebrew, ejecuten:

Shell

```
brew install openjdk@17
```

Posteriormente, vinculen la instalación para que sea detectable por el sistema:

Shell

```
sudo ln -sf /opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk \
/Library/Java/JavaVirtualMachines/openjdk-17.jdk
```

Para corroborar el éxito del proceso:

Shell

```
/usr/libexec/java_home -V
```

El listado debe incluir la versión 17.

2. Ejecución con Java 17

Dentro del directorio del laboratorio:

Shell

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) \
PATH="$JAVA_HOME/bin:$PATH" \
SBT_OPTS="--add-exports=java.base/sun.nio.ch=ALL-UNNAMED" \
sbt run
```

Este procedimiento asegura que:

- Se emplee Java 17 únicamente para la tarea actual.
- Se mantenga la versión global del entorno sin alteraciones.
- Se incorporen los flags necesarios para evitar fallos internos en el manejo de memoria de Hadoop.

Configuración sugerida en **build.sbt**

Para mitigar conflictos de classloaders y asegurar un cierre limpio de Spark, integren lo siguiente:

None

```
ThisBuild / classLoaderLayeringStrategy :=  
ClassLoaderLayeringStrategy.Flat  
  
fork := true  
  
ThisBuild / javaOptions ++= Seq(  
  "--add-exports=java.base/sun.nio.ch=ALL-UNNAMED",  
  "--add-opens=java.base/java.nio=ALL-UNNAMED"  
)
```

Estas opciones minimizan los problemas de interoperabilidad entre Spark, Hadoop y los runtimes de Java modernos.

Modo local

No requiere levantar ningún proceso adicional. Spark simula el cluster usando todos los cores de la máquina. El parámetro `.master("local[*]")` en la `SparkSession` activa este modo. Es suficiente para desarrollar y probar todos los ejercicios.

Modo cluster (simulado)

Para simular un cluster real en una sola máquina:

1. Iniciar el Master: `$SPARK_HOME/sbin/start-master.sh` desde `${SPARK_FOLDER}`.
2. Iniciar un Worker: `$SPARK_HOME/sbin/start-worker.sh spark://localhost:7077 -m 1G -c 1`.
3. Verificar el estado del cluster en `http://localhost:8080`.

4. Reemplazar `.master("local[*"])` por `.master("spark://localhost:7077")` en la `SparkSession`.

Links

- Link al esqueleto
<https://drive.google.com/file/d/1KDHGneyqXoXKMrrjVfqs8wPYUHrrL5nN/view?usp=sharing>
- Link al mock del servidor
https://drive.google.com/file/d/1LOGKui_4FsxTGCQ2EAWllwRW23kHEQZI/view?usp=sharing
- Link al hello world
<https://drive.google.com/file/d/1uV37eVtUp2uin3XHVljylX6p2WvVVSbj/view?usp=sharing>

Entregables

- Código fuente completo en el repositorio GitHub del grupo, en Scala.
 - El nombre del repositorio debe seguir el formato paradigmas26-lab3-gXX
 - Puede incluir cualquier modificación al código del esqueleto.
 - No es necesario entregar test funcionando.
 - El repositorio no debe contener el código de hello_world ni reddit-mock
 - El repositorio no puede tener una carpeta llamada skeleton, sino que en su raíz ya contiene el `README.md`, `build.sbt`, directorios `src` y `data`.
 - El primer commit del repositorio debe contener el código del esqueleto original, aunque luego decidan implementar el laboratorio desde 0. Dicho commit debe tener tag `v0-skeleton`
 - El commit con la entrega final del laboratorio debe tener tag `v1`
- `Makefile` que compile y ejecute el programa con un único comando.
- `README.md` con instrucciones para configurar el entorno y ejecutar el programa en modo local.

- **INFORME.md** con las respuestas a todos los puntos marcados en los ejercicios, más las siguientes secciones:
 - Decisiones de diseño relevantes: qué modificaron respecto a los laboratorios anteriores y por qué.
 - Respuesta a las preguntas conceptuales de cada ejercicio.
- NO HACER git add .
- No entregar archivos que comienzan con . _

Puntos extra

- **Extender la batería de tests** para que funcionen correctamente con el output de Spark y tengan mayor cobertura sobre todos los requerimientos de laboratorio.
- **Comparativa de rendimiento:** medir y comparar el tiempo de ejecución entre la versión secuencial (esqueleto) y la distribuida (Lab 3) variando el número de feeds. Incluir una tabla o gráfico en **Informe.md**.
- **Persistencia intermedia:** guardar los posts descargados a disco antes del cómputo de NER, simulando el caso donde los datos no caben en memoria. Discutir el impacto en el pipeline.